

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ I



BÁO CÁO BÀI TẬP LỚN
MÔN: KỸ THUẬT VI XỬ LÝ (ELE1317)

BÀI 1: HỆ THỐNG GHI NHIỆT ĐỘ – ĐỘ ẨM VÀ GHI/PHÁT ÂM
THANH
TRÊN THẺ NHỚ SD DÙNG VI ĐIỀU KHIỂN STM32F103C8T6

Giảng viên hướng dẫn:	
Sinh viên thực hiện:	
Mã sinh viên:	
Lớp / Nhóm:	

Hà Nội, tháng 10/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ BÀI.....	4
1.1. Yêu cầu.....	4
1.2. Mục tiêu và phạm vi.....	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ LINH KIỆN.....	4
2.1. Vi điều khiển STM32F103C8T6.....	4
2.2. Cảm biến DHT11.....	4
2.3. Thẻ nhớ SD ở chế độ SPI và hệ thống file FAT.....	5
2.4. Module micro MAX9814.....	5
2.5. Định dạng file WAV.....	6
2.6. Phát âm thanh bằng PWM.....	6
2.7. Danh sách linh kiện.....	7
CHƯƠNG 3. THIẾT KẾ PHẦN CỨNG.....	7
3.1. Sơ đồ khối.....	7
3.2. Bảng phân bố chân.....	7
3.3. Sơ đồ mạch mô phỏng trên Proteus.....	8
CHƯƠNG 4. THIẾT KẾ PHẦN MỀM.....	8
4.1. Cấu trúc chương trình.....	8
4.2. Bài 1a – Đọc DHT11 và ghi DATA.CSV.....	9
4.3. Bài 1b – Ghi âm ra file WAV.....	9
4.4. Bài 1c – Phát lại file WAV.....	10
CHƯƠNG 5. MÔ PHỎNG VÀ KẾT QUẢ.....	10
5.1. Chuẩn bị mô phỏng.....	10
5.2. Kết quả bài 1a.....	10
5.3. Kết quả bài 1b.....	11
5.4. Kết quả bài 1c.....	12
5.5. Tổng hợp.....	12
CHƯƠNG 6. KHÓ KHĂN VÀ CÁCH KHẮC PHỤC.....	12
CHƯƠNG 7. KẾT LUẬN.....	13
7.1. Kết quả đạt được.....	13
7.2. Hạn chế và hướng phát triển.....	13
TÀI LIỆU THAM KHẢO.....	14

DANH MỤC HÌNH ẢNH

Hình 1. Board STM32F103C8T6 (Blue Pill).....	4
Hình 2. Module cảm biến DHT11.....	5
Hình 3. Module thẻ nhớ micro SD giao tiếp SPI.....	5
Hình 4. Module micro MAX9814.....	6
Hình 5. Sơ đồ khối hệ thống.....	7
Hình 6. Toàn bộ sơ đồ mạch Proteus (chụp màn hình đầy đủ khi chưa chạy, phóng to vừa trang).....	8
Hình 7. Kết quả chạy mô phỏng: Virtual Terminal hiển thị dữ liệu DHT11, ghi âm và phát lại.....	11
Hình 8. Cửa sổ 7-Zip mở sdcard.img, thấy DATA.CSV, REC01.WAV, PLAY.WAV	11
Hình 9. File DATA.CSV mở bằng Excel (cột Thoi_gian_s, Nhiệt_do_C, Do_am_%)	11
Hình 10. REC01.WAV mở trong Windows Media Player / Audacity (dạng sóng).....	12
Hình 11. (Tuỳ chọn) Oscilloscope/Graph trong Proteus: tín hiệu PWM tại PB6 hoặc tín hiệu sau lọc ở loa.....	12

DANH MỤC BẢNG BIỂU

Bảng 1. Cấu trúc header file WAV 44 byte.....	6
Bảng 2. Danh sách linh kiện.....	7
Bảng 3. Phân bổ chân vi điều khiển.....	8
Bảng 4. Các file mã nguồn.....	8
Bảng 5. Tổng hợp kết quả.....	12
Bảng 6. Các vấn đề gặp phải khi mô phỏng.....	13

CHƯƠNG 1. GIỚI THIỆU ĐỀ BÀI

1.1. Yêu cầu

Đề bài 1 yêu cầu thiết kế một hệ thống nhúng dựa trên vi điều khiển STM32F103C8T6 với ba chức năng:

- a) Kết nối module thẻ nhớ SD và cảm biến DHT11; cứ 5 giây đọc nhiệt độ, độ ẩm một lần và lưu vào thẻ SD. Kiểm tra dữ liệu bằng đầu đọc thẻ USB trên máy tính.
- b) Dùng module micro MAX9814, đọc tín hiệu qua bộ ADC, ghi âm giọng nói thành file .WAV lưu trên thẻ SD.
- c) Thiết kế mạch loa và phát lại file .WAV đã lưu trên thẻ SD.

1.2. Mục tiêu và phạm vi

Hệ thống được viết bằng ngôn ngữ C ở mức thanh ghi (bare-metal, không dùng HAL) để thể hiện rõ cách cấu hình các ngoại vi GPIO, Timer, ADC, PWM, USART và giao tiếp SPI. Hệ thống file FAT trên thẻ được quản lý bằng thư viện mã nguồn mở FatFs. Toàn bộ thiết kế được kiểm chứng bằng phần mềm mô phỏng Proteus 8 Professional, đồng thời cung cấp một bản firmware riêng cho mạch thật.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ LINH KIỆN

2.1. Vi điều khiển STM32F103C8T6

STM32F103C8T6 (board "Blue Pill") dùng lõi ARM Cortex-M3 32 bit, tần số tối đa 72 MHz, 64 KB Flash, 20 KB SRAM, có 2 bộ ADC 12 bit, 4 timer 16 bit, 2 SPI, 3 USART. Trong đề tài, chip chạy ở 64 MHz từ bộ dao động nội HSI qua PLL.



Hình 1. Board STM32F103C8T6 (Blue Pill)

2.2. Cảm biến DHT11

DHT11 đo nhiệt độ 0–50 °C (± 2 °C) và độ ẩm 20–90 %RH (± 5 %), giao tiếp một dây. Vi điều khiển kéo đường DATA xuống mức thấp ≥ 18 ms để bắt đầu; cảm biến phản hồi rồi gửi 40 bit: độ ẩm nguyên, độ ẩm thập phân, nhiệt độ nguyên, nhiệt độ thập

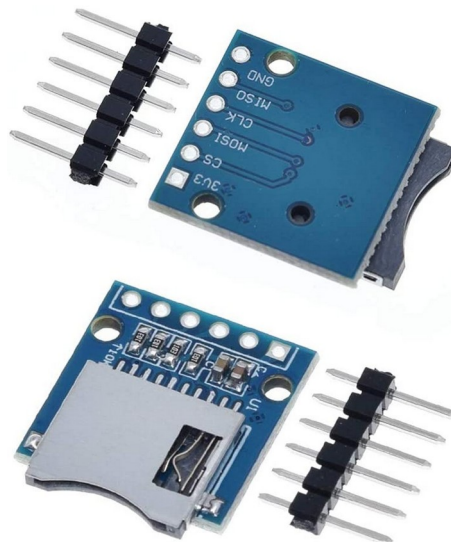
phân và checksum. Mỗi bit bắt đầu bằng mức thấp 50 μ s, sau đó mức cao 26–28 μ s là bit 0 hoặc 70 μ s là bit 1.



Hình 2. Module cảm biến DHT11

2.3. Thẻ nhớ SD ở chế độ SPI và hệ thống file FAT

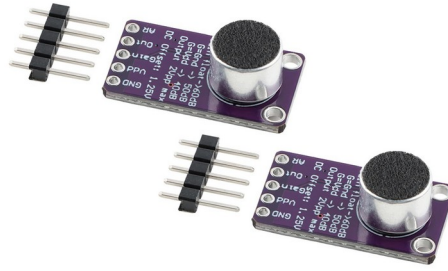
Thẻ SD hỗ trợ chế độ SPI với 4 dây CS, SCK, MOSI, MISO. Quá trình khởi tạo gồm: gửi ≥ 74 xung clock khi CS ở mức cao, lệnh CMD0 (về trạng thái idle), CMD8 (kiểm tra điện áp), ACMD41 (khởi tạo), CMD58 (đọc OCR, xác định thẻ SDHC). Sau đó đọc/ghi từng khối 512 byte bằng CMD17/CMD24. Thư viện FatFs đảm nhiệm cấu trúc FAT16/FAT32 để máy tính đọc được file trực tiếp.



Hình 3. Module thẻ nhớ micro SD giao tiếp SPI

2.4. Module micro MAX9814

MAX9814 tích hợp micro electret và bộ khuếch đại có điều khiển độ lợi tự động (AGC), ngõ ra có mức DC khoảng 1,25 V, biên độ tối đa 2 Vpp, phù hợp đưa thẳng vào ADC 0–3,3 V của STM32.



Hình 4. Module micro MAX9814

2.5. Định dạng file WAV

File WAV (PCM) gồm phần header 44 byte và phần dữ liệu mẫu. Đề tài dùng định dạng PCM 8 bit, mono, không dấu (giá trị 128 tương ứng mức 0).

Offset	Kích thước	Trường	Giá trị
0	4	ChunkID	"RIFF"
4	4	ChunkSize	36 + số byte dữ liệu
8	4	Format	"WAVE"
12	4	Subchunk1ID	"fmt "
16	4	Subchunk1Size	16
20	2	AudioFormat	1 (PCM)
22	2	NumChannels	1
24	4	SampleRate	8000 (thật) / 4000 (Proteus)
28	4	ByteRate	$= \text{SampleRate} \times 1$
32	2	BlockAlign	1
34	2	BitsPerSample	8
36	4	Subchunk2ID	"data"
40	4	Subchunk2Size	số byte dữ liệu

Bảng 1. Cấu trúc header file WAV 44 byte

2.6. Phát âm thanh bằng PWM

STM32F103 không có bộ DAC, vì vậy âm thanh được tạo bằng điều chế độ rộng xung (PWM) tần số cao (250 kHz, 8 bit). Độ rộng xung bằng giá trị mẫu; mạch lọc thông thấp RC loại bỏ thành phần tần số sóng mang, khôi phục tín hiệu âm thanh và đưa tới mạch khuếch đại PAM8403 để kéo loa.

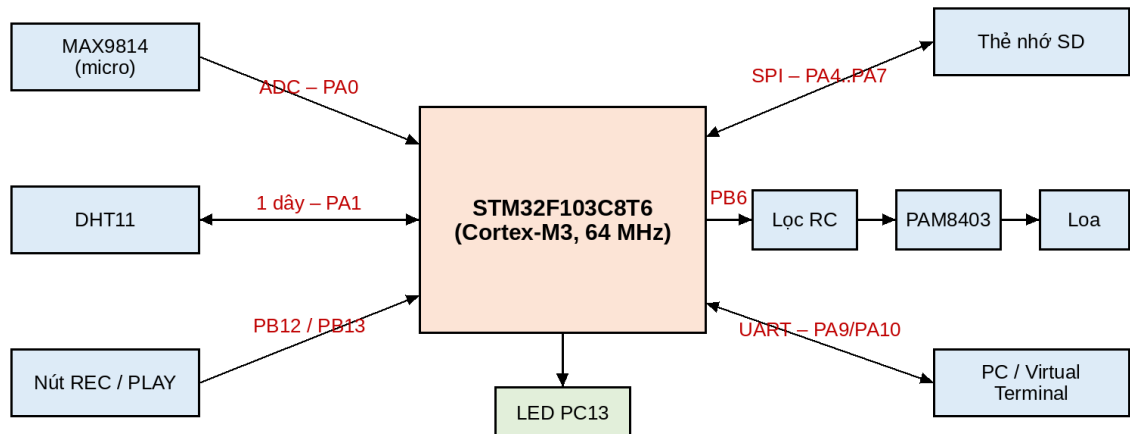
2.7. Danh sách linh kiện

STT	Linh kiện	SL	Chức năng
1	STM32F103C8T6 Blue Pill	1	Vi điều khiển trung tâm
2	Module micro SD (SPI) + thẻ ≤ 32 GB	1	Lưu DATA.CSV và file WAV
3	Cảm biến DHT11	1	Đo nhiệt độ, độ ẩm
4	Module micro MAX9814	1	Thu âm thanh
5	Module khuếch đại PAM8403 + loa 8 Ω 3 W	1	Phát âm thanh
6	Nút nhấn 6×6, LED, điện trở, tụ	—	REC/PLAY, mạch lọc RC
7	ST-Link V2	1	Nạp chương trình
8	USB–TTL CH340	1	Xem log UART trên máy tính
9	Đầu đọc thẻ USB	1	Kiểm tra dữ liệu trên PC
10	Breadboard, dây cắm, nguồn 5 V	—	Lắp ráp

Bảng 2. Danh sách linh kiện

CHƯƠNG 3. THIẾT KẾ PHẦN CỨNG

3.1. Sơ đồ khối



Hình 5. Sơ đồ khối hệ thống

3.2. Bảng phân bổ chân

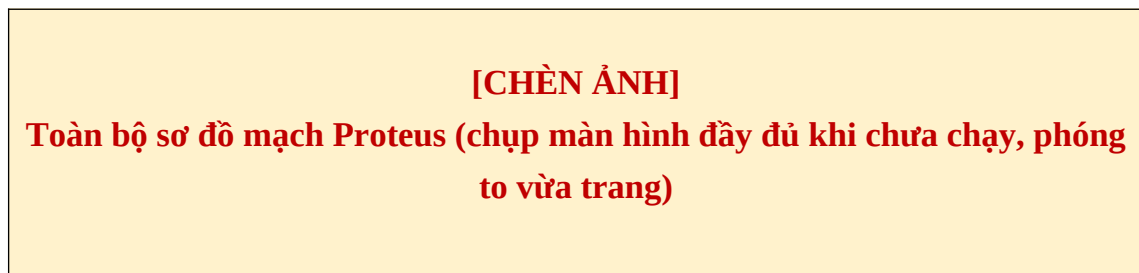
Chân STM32	Ngoại vi	Kết nối
PA0	ADC1_IN0	Ngõ ra MAX9814 (MIC)
PA1	TIM2_CH2 (Input Capture)	DHT11 DATA, điện trở kéo lên 10 kΩ
PA4	GPIO	SD – CS

Chân STM32	Ngoại vi	Kết nối
PA5	GPIO (SCK)	SD – CLK
PA6	GPIO (MISO)	SD – DO
PA7	GPIO (MOSI)	SD – DI
PA9 / PA10	USART1 TX / RX	Virtual Terminal / CH340, 115200 baud
PB6	TIM4_CH1 (PWM)	R 330 Ω $\rightarrow$ C $\rightarrow$ loa / PAM8403
PB12	GPIO in, pull-up	Nút REC (ghi âm)
PB13	GPIO in, pull-up	Nút PLAY (phát)
PC13	GPIO out	LED báo trạng thái

Bảng 3. Phân bổ chân vi điều khiển

3.3. Sơ đồ mạch mô phỏng trên Proteus

Mạch mô phỏng gồm: U1 STM32F103C8, U2 DHT11, MMC1 thẻ nhớ (gắn file ảnh thẻ sdcard.img dung lượng 32 MB định dạng FAT16), Virtual Terminal nối USART1, nguồn tín hiệu MIC (file âm thanh mic_input.wav) đưa vào PA0, nút nhấn ở PB12/PB13 và loa LS1 nối PB6 qua R2, C2.



Hình 6. Toàn bộ sơ đồ mạch Proteus (chụp màn hình đầy đủ khi chưa chạy, phóng to vừa trang)

CHƯƠNG 4. THIẾT KẾ PHẦN MỀM

4.1. Cấu trúc chương trình

File	Nội dung
startup.c, link.ld	Bảng vector ngắt, khởi tạo RAM, bản đồ bộ nhớ
stm32f103.h	Định nghĩa thanh ghi ngoại vi
main.c	Clock, GPIO, Timer, ADC, UART, đọc DHT11, ghi/phát WAV, vòng lặp chính
sd_spi.c	Driver thẻ SD qua SPI (diskio cho FatFs)
fatfs/	Thư viện FatFs R0.11 (hệ thống file FAT)

Bảng 4. Các file mã nguồn

Vòng lặp chính hoạt động theo kiểu siêu vòng lặp (super-loop) với các tác vụ không chặn: mỗi 5 s gọi hàm log_dht(); nếu nhấn PB12 thì gọi do_record(); nếu nhấn PB13 thì gọi do_play(). Bộ đếm thời gian ms_ticks được tăng trong ngắt TIM1 mỗi 1 ms.

```
for(;;){
    if(ms_ticks - last >= LOG_PERIOD_MS){ /* mỗi 5000 ms */
        last = ms_ticks; log_dht(); }
    if(!(GPIOB->IDR & (1u<<12))){ /* nút REC */
        delay_ms(30); if(!(GPIOB->IDR & (1u<<12))) do_record(); }
    if(!(GPIOB->IDR & (1u<<13))){ /* nút PLAY */
        delay_ms(30); if(!(GPIOB->IDR & (1u<<13))) do_play(last_wav); }
}
```

4.2. Bài 1a – Đọc DHT11 và ghi DATA.CSV

Để đo chính xác độ rộng các xung vài chục micro giây, chương trình dùng chức năng Input Capture của TIM2 kênh 2 (PA1) với độ phân giải 1 μ s: mỗi sườn xuống, phần cứng tự chốt giá trị bộ đếm vào CCR2. Khoảng cách giữa hai sườn xuống liên tiếp là $50 + 26 \mu\text{s} \approx 76 \mu\text{s}$ (bit 0) hoặc $50 + 70 \mu\text{s} \approx 120 \mu\text{s}$ (bit 1), nên ngưỡng phân biệt được chọn là 110 μ s.

```
GPIOA->BRR = 1u<<1; delay_ms(dht_start_ms); /* kéo xuống: xung start */
GPIOA->BSRR = 1u<<1;
TIM2->CCMR1 = (TIM2->CCMR1 & ~(0xFFu<<8)) | (1u<<8); /* IC2 <- TI2 */
TIM2->CCER |= (1u<<5) | (1u<<4); /* bắt sườn xuống */
while(dht_n < 42){ /* chốt 42 sườn xuống */
    if(TIM2->SR & (1u<<2)) dht_ts[dht_n++] = TIM2->CCR2;
}
for(i=0;i<40;i++){ /* giải mã 40 bit */
    p = dht_ts[i+1] - dht_ts[i];
    d[i/8] = (d[i/8]<<1) | (p > 110); /* >110 us -> bit 1 */
}
if((uint8_t)(d[0]+d[1]+d[2]+d[3]) != d[4]) return -4; /* sai checksum */
*h = d[0]; *t = d[2];
```

Sau khi đọc thành công, chương trình mở file DATA.CSV ở chế độ ghi tiếp (FA_OPEN_ALWAYS + f_lseek tới cuối file), nếu file rỗng thì ghi dòng tiêu đề "Thoi_gian_s,Nhiet_do_C,Do_am_%", rồi ghi thêm một dòng dữ liệu và đóng file (f_close) để dữ liệu được lưu ngay, không bị mất khi rút nguồn.

4.3. Bài 1b – Ghi âm ra file WAV

Timer TIM3 tạo ngắt với tần số lấy mẫu (8 kHz trên mạch thật). Trong ngắt, giá trị ADC 12 bit được lấy 8 bit cao (ADC1->DR >> 4) làm mẫu PCM không dấu, rồi khởi động lần chuyển đổi tiếp theo. Trên mạch thật, chương trình dùng bộ đệm đôi 2×512 byte: ngắt ghi vào một nửa trong khi vòng lặp chính ghi nửa còn lại xuống thẻ. Khi kết thúc, header WAV được ghi lại ở đầu file với kích thước dữ liệu thực.

```

void TIM3_IRQHandler(void){ /* tần số = SAMPLE_RATE */
    TIM3->SR = 0;
    if(mode==1){ /* ghi âm */
        abuf[a_cur][a_pos++] = (uint8_t)(ADC1->DR >> 4); /* 12 bit -> 8 bit */
        ADC1->CR2 |= 1u<<22; /* bắt đầu chuyển đổi mới */
    } else { /* phát */
        TIM4->CCR1 = abuf[a_cur][a_pos++]; /* độ rộng xung PWM */
    }
    if(a_pos >= 512){ a_pos = 0; a_ready = a_cur; a_cur ^= 1; } /* đổi bộ đệm */
}

```

File được đặt tên tự động REC01.WAV, REC02.WAV, ... (chương trình tìm số thứ tự chưa tồn tại trên thẻ).

4.4. Bài 1c – Phát lại file WAV

Chương trình đọc header để lấy tần số lấy mẫu, cấu hình lại chu kỳ TIM3, sau đó đọc dữ liệu từ thẻ vào bộ đệm; ngắt TIM3 lần lượt nạp từng mẫu vào thanh ghi so sánh TIM4->CCR1 để thay đổi độ rộng xung PWM tại PB6 (TIM4 chạy $64\text{ MHz}/256 = 250\text{ kHz}$). Khi phát xong, CCR1 được đặt về 128 (mức giữa) để loa không bị lệch DC.

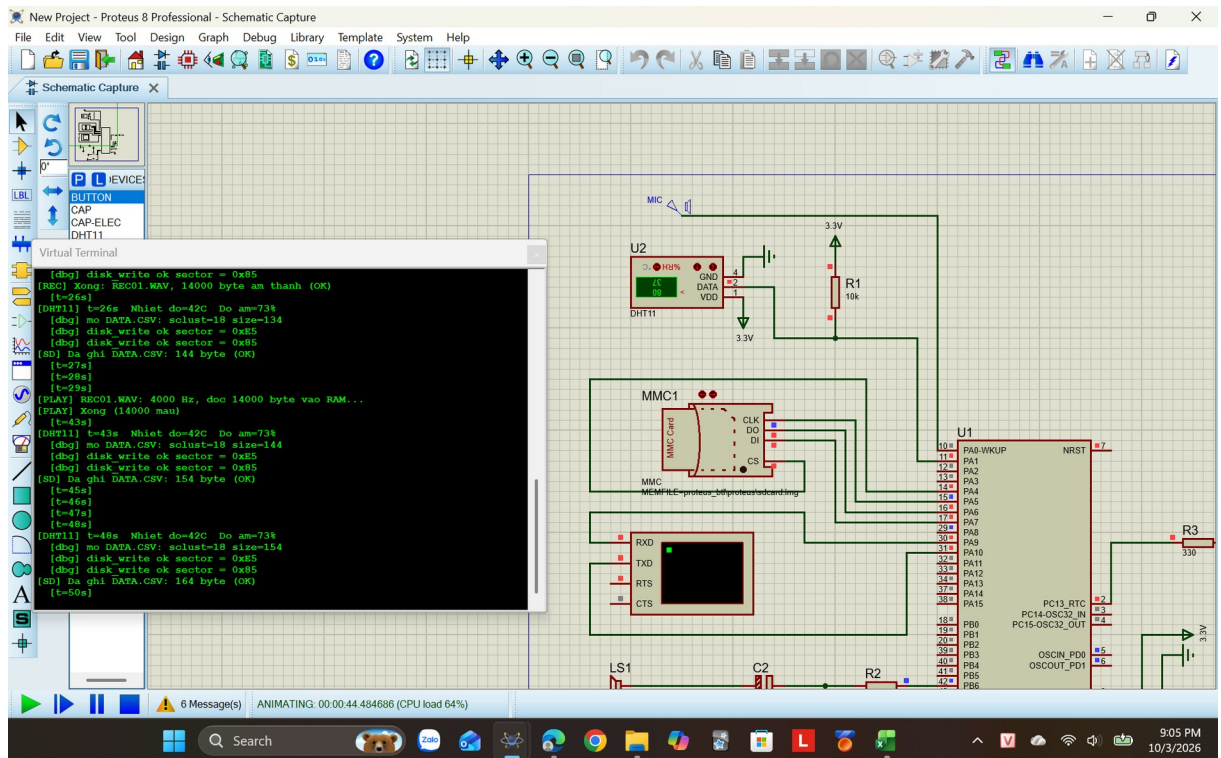
CHƯƠNG 5. MÔ PHỎNG VÀ KẾT QUẢ

5.1. Chuẩn bị mô phỏng

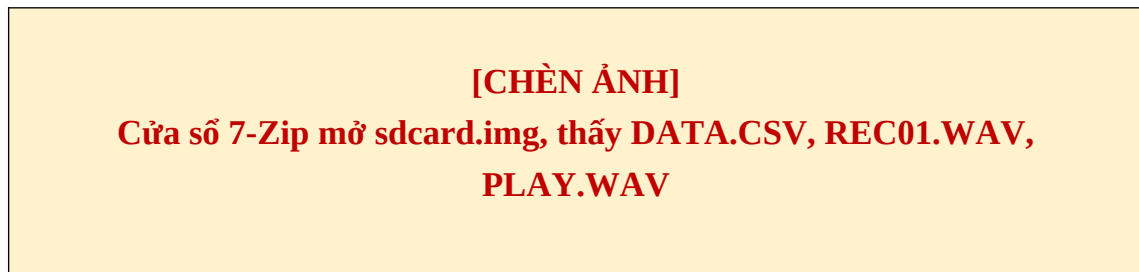
- Nạp file btl_ktvxl_PROTEUS.hex vào U1 (Program File), tần số 64 MHz.
- MMC1: Card Image File = sdcard.img (FAT16, 32 MB).
- Virtual Terminal: 115200 baud, 8N1.
- MIC: nguồn âm thanh từ file mic_input.wav.

5.2. Kết quả bài 1a

Sau khi chạy, Virtual Terminal hiển thị "[SD] Mount OK", sau đó cứ 5 giây xuất hiện một dòng đọc DHT11 và thông báo ghi DATA.CSV thành công; kích thước file tăng dần $134 \rightarrow 144 \rightarrow 154 \rightarrow 164$ byte, mỗi dòng 10 byte. Khi thay đổi giá trị nhiệt độ/độ ẩm trên DHT11, các giá trị ghi vào thẻ thay đổi theo.



Hình 7. Kết quả chạy mô phỏng: Virtual Terminal hiển thị dữ liệu DHT11, ghi âm và phát lại Sau khi dừng mô phỏng, mở file sdcard.img bằng 7-Zip (tương đương cắm thẻ vào đầu đọc USB) thấy các file DATA.CSV và REC01.WAV. Mở DATA.CSV bằng Excel cho kết quả đúng định dạng bảng.



Hình 8. Cửa sổ 7-Zip mở sdcard.img, thấy DATA.CSV, REC01.WAV, PLAY.WAV



Hình 9. File DATA.CSV mở bằng Excel (cột Thoi_gian_s, Nhiet_do_C, Do_am_%)

5.3. Kết quả bài 1b

Nhấn nút PB12, terminal hiển thị "[REC] Dang ghi REC01.WAV (4000 Hz, 3500 ms)", "Da thu 14000 mau" và "[REC] Xong: REC01.WAV, 14000 byte am thanh (OK)". Số mẫu đúng bằng $4000 \text{ mẫu/s} \times 3,5 \text{ s} = 14000$, chứng tỏ timer lấy mẫu chạy đúng tần số. File REC01.WAV có kích thước 14044 byte (44 byte header + 14000 byte dữ liệu), mở được bằng trình phát nhạc trên Windows.

[CHÈN ẢNH]
REC01.WAV mở trong Windows Media Player / Audacity (dạng sóng)

Hình 10. REC01.WAV mở trong Windows Media Player / Audacity (dạng sóng)

5.4. Kết quả bài 1c

Nhấn nút PB13, terminal hiển thị "[PLAY] REC01.WAV: 4000 Hz, đọc 14000 byte vào RAM..." và "[PLAY] Xong (14000 mẫu)". Tín hiệu PWM tại PB6 được lọc và đưa ra loa LS1. Trong quá trình ghi âm và phát lại, việc ghi DHT11 vẫn tiếp tục bình thường ngay sau đó ($t = 43 \text{ s}, 48 \text{ s}$).

[CHÈN ẢNH]
(Tuỳ chọn) Oscilloscope/Graph trong Proteus: tín hiệu PWM tại PB6 hoặc tín hiệu sau lọc ở loa

Hình 11. (Tuỳ chọn) Oscilloscope/Graph trong Proteus: tín hiệu PWM tại PB6 hoặc tín hiệu sau lọc ở loa

5.5. Tổng hợp

Yêu cầu	Kết quả	Đánh giá
1a. Ghi nhiệt độ, độ ẩm mỗi 5 s	DATA.CSV ghi đều mỗi 5 s, đọc được bằng Excel	Đạt
1b. Ghi âm ra file WAV	REC01.WAV 14000 mẫu, header hợp lệ	Đạt
1c. Phát lại file WAV	Phát đủ 14000 mẫu qua PWM PB6	Đạt

Bảng 5. Tổng hợp kết quả

CHƯƠNG 6. KHÓ KHĂN VÀ CÁCH KHẮC PHỤC

Trong quá trình mô phỏng, nhóm gặp một số hạn chế của mô hình STM32 trong Proteus và đã phân tích, khắc phục như sau:

Hiện tượng	Nguyên nhân	Cách khắc phục
Thẻ SD không khởi tạo, mọi byte nhận về là 0x00	Khối SPI1 phần cứng của mô hình STM32 trong Proteus không trả dữ liệu đúng	Viết SPI bằng phần mềm (bit-bang) trên PA5/PA6/PA7
Delay và chu kỳ 5 s dài gấp ~8 lần thực tế, trong khi UART vẫn đúng	Mô hình timer chạy theo clock 8 MHz thay vì 64 MHz sau PLL	Bản Proteus (-DPROTEUS) tính hệ số timer theo 8 MHz

Hiện tượng	Nguyên nhân	Cách khắc phục
Đọc DHT11 lỗi do vòng lặp kiểm tra mức logic quá chậm	CPU mô phỏng không đủ nhanh để lấy mẫu chân ở mức μs	Dùng Input Capture TIM2_CH2, phần cứng tự chốt thời điểm sườn xung
f_open/f_write báo FR_INT_ERR dù thẻ và file hệ thống đúng (kiểm chứng bằng QEMU)	Proteus thực thi sai khối lệnh IT nhiều lệnh (ITTTE + MLA) do trình biên dịch tối ưu sinh ra trong hàm clust2sect	Biên dịch riêng FatFs với -O0
Ghi âm bị tràn bộ đệm (overflow), chỉ ghi được 1088 byte	Tốc độ ghi thẻ trong mô phỏng không theo kịp lấy mẫu 8 kHz	Bản Proteus thu 4 kHz vào RAM (14000 byte) rồi mới ghi thẻ; bản mạch thật giữ nguyên 8 kHz dùng bộ đệm đôi

Bảng 6. Các vấn đề gặp phải khi mô phỏng

Những vấn đề này chỉ xuất hiện trong môi trường mô phỏng. Vì vậy mã nguồn được tổ chức với tùy chọn biên dịch PROTEUS để tạo hai bản firmware: btl_ktvxl_PROTEUS.hex cho mô phỏng và btl_ktvxl_MACH_THAT.hex cho mạch thật.

CHƯƠNG 7. KẾT LUẬN

7.1. Kết quả đạt được

- Xây dựng hoàn chỉnh hệ thống trên STM32F103C8T6 lập trình ở mức thanh ghi, đáp ứng đủ ba yêu cầu 1a, 1b, 1c.
- Nắm được cách cấu hình GPIO, Timer (ngắt, Input Capture, PWM), ADC, USART, giao thức SD-SPI, hệ thống file FAT và định dạng WAV.
- Mô phỏng thành công trên Proteus; dữ liệu trên thẻ đọc được bằng máy tính.

7.2. Hạn chế và hướng phát triển

- Âm thanh 8 bit, tần số lấy mẫu thấp nên chất lượng ở mức giọng nói; có thể nâng lên 16 bit, dùng DMA cho ADC và SPI.
- Chưa có đồng hồ thời gian thực; có thể thêm DS3231 để ghi ngày giờ thực vào DATA.CSV.
- Có thể bổ sung màn hình LCD hiển thị nhiệt độ, độ ẩm và trạng thái ghi/phát.

TÀI LIỆU THAM KHẢO

- [1] STMicroelectronics, RM0008 Reference Manual – STM32F101xx/F102xx/F103xx.
- [2] STMicroelectronics, STM32F103x8/xB Datasheet.
- [3] Aosong, DHT11 Humidity & Temperature Sensor Datasheet.
- [4] Maxim Integrated, MAX9814 Microphone Amplifier with AGC Datasheet.
- [5] SD Association, SD Specifications Part 1 – Physical Layer Simplified Specification.
- [6] ChaN, FatFs – Generic FAT Filesystem Module, <http://elm-chan.org/fsw/ff/>
- [7] rubin-khadka, STM32_MultiSensor_MicroSD_Datalogger, GitHub.
- [8] Gaohzc414, stm32-portable-audio-recorder, GitHub.
- [9] Bài giảng Kỹ thuật vi xử lý, Học viện Công nghệ Bưu chính Viễn thông.